30-DAYS-OF-DATA-ANALYSIS-CHALLENGE | PYTHON · MYSQL · EXCEL

Pandas Reference Notes

Day 01 – Day 05 · Explanations, examples and outputs

01 DAY 01
Pandas Introduction and DataFrame Basics

02 DAY 02
Data Selection and Filtering

03 DAY 03
DataFrame Operations

04 DAY 04
GroupBy and Aggregation

05 DAY 05
Sorting and Data Analysis

Beginner-friendly reference for learners · Every example uses one sample employee dataset

Beginner-friendly reference notes covering the first 5 days of the challenge (Python, MySQL, Excel).

Every concept has a simple explanation, a code example, and the expected output.

How to read this guide

`df.head()` Blue bar: Python code	`8 rows x 6 cols` Green bar: output	**Tip:** helpful shortcut Blue box	**Warning:** common mistake Orange box

Table of Contents

Sample Dataset Used in All Examples 4

Day 01 – Pandas Introduction and DataFrame Basics 5

1.1 What is Pandas? 5
1.2 Installing and Importing 5
1.3 The Two Core Data Structures 5
1.4 Creating a DataFrame 5
1.5 Inspecting a DataFrame 6
1.6 Key Points to Remember 7

Day 02 – Data Selection and Filtering 8

2.1 Selecting Columns 8
2.2 Selecting Rows and Columns with loc and iloc 8
2.3 Filtering with Conditions (Boolean Masking) 8
2.4 Multiple Conditions 9
2.5 Useful Filtering Methods 9
2.6 Setting a Column as Index (Optional but Handy) 9
2.7 Key Points to Remember 10

Day 03 – DataFrame Operations 11

3.1 Adding New Columns 11
3.2 Renaming Columns 11
3.3 Dropping Columns and Rows 11
3.4 inplace vs Assigning Back 11
3.5 Handling Missing Values 11
3.6 Handling Duplicates 12
3.7 Changing Data Types 12
3.8 Applying Functions: apply, map, replace 12
3.9 String Operations 13
3.10 Resetting and Setting Index 13
3.11 Key Points to Remember 13

Day 04 – GroupBy and Aggregation . . . 14

4.1 The Idea: Split → Apply → Combine . . . 14
4.2 Basic GroupBy . . . 14
4.3 Grouping by Multiple Columns . . . 15
4.4 Multiple Aggregations with agg() . . . 15
4.5 Turning the Result Back into a Normal DataFrame . . . 15
4.6 Filtering Before and After GroupBy . . . 16
4.7 transform(): Keep the Original Shape . . . 16
4.8 Pivot Tables . . . 16
4.9 crosstab(): Count Combinations . . . 16
4.10 SQL ↔ Pandas Comparison . . . 16
4.11 Key Points to Remember . . . 16

Day 05 – Sorting and Data Analysis . . . 18

5.1 Sorting by Values . . . 18
5.2 Sorting by Index . . . 18
5.3 Top N and Bottom N . . . 18
5.4 Ranking . . . 18
5.5 Counting Values . . . 19
5.6 Summary Statistics . . . 19
5.7 Finding the Row of Max / Min . . . 19
5.8 Correlation . . . 19
5.9 Cumulative and Percentage Calculations . . . 20
5.10 Mini Analysis Workflow (Putting Days 1–5 Together) . . . 20
5.11 Key Points to Remember . . . 20

Quick Cheat Sheet . . . 21

Practice Questions . . . 21

About This Repository . . . 22

Sample Dataset Used in All Examples

Copy this once and run every example below on it.

```
import pandas as pd
import numpy as np
data = {
    "emp_id":     [101, 102, 103, 104, 105, 106, 107, 108],
    "name":       ["Asha", "Ravi", "Meera", "Kiran", "Sneha", "Arjun", "Divya", "Rahul"],
    "department": ["IT", "HR", "IT", "Sales", "Sales", "HR", "IT", "Sales"],
    "city":       ["Bengaluru", "Pune", "Chennai", "Pune", "Bengaluru", "Chennai", "Pune", "Bengalur
    u"],
    "age":        [28, 34, 25, 31, 29, 38, 26, 33],
    "salary":     [60000, 45000, 55000, 40000, 42000, 52000, 65000, 48000]
}
df = pd.DataFrame(data)
print(df)
```

```
   emp_id   name department       city  age  salary
0     101   Asha         IT  Bengaluru   28   60000
1     102   Ravi         HR       Pune   34   45000
2     103  Meera         IT    Chennai   25   55000
3     104  Kiran      Sales       Pune   31   40000
4     105  Sneha      Sales  Bengaluru   29   42000
5     106  Arjun         HR    Chennai   38   52000
6     107  Divya         IT       Pune   26   65000
7     108  Rahul      Sales  Bengaluru   33   48000
```

DAY 01

Pandas Introduction and DataFrame Basics

1.1 What is Pandas?

Pandas is a Python library for **data analysis and data manipulation**. It works with tabular data (rows and columns) like an Excel sheet or a SQL table, but with the power of Python.

Why use it?

- Handles large datasets quickly
- Reads and writes CSV, Excel, SQL, JSON
- Cleans, filters, groups and summarizes data in a few lines

1.2 Installing and Importing

```
pip install pandas
```

```
import pandas as pd      # 'pd' is the standard short name
import numpy as np       # often used together with pandas
```

1.3 The Two Core Data Structures

Structure	Dimension	Looks like
Series	1-D	A single column
DataFrame	2-D	A full table (many columns)

Series

A Series is a one-dimensional labeled array.

```
marks = pd.Series([80, 90, 75], index=["Maths", "Science", "English"])
print(marks)
```

```
Maths      80
Science    90
English    75
dtype: int64
```

DataFrame

A DataFrame is a collection of Series that share the same index. Each column is a Series.

```
type(df)           # pandas.core.frame.DataFrame
type(df["age"])    # pandas.core.series.Series
```

1.4 Creating a DataFrame

From a dictionary (most common; keys become column names):

```
df = pd.DataFrame({"name": ["Asha", "Ravi"], "age": [28, 34]})
```

From a list of lists (you provide the column names):

```
df = pd.DataFrame([["Asha", 28], ["Ravi", 34]], columns=["name", "age"])
```

From a CSV or Excel file:

```
df = pd.read_csv("employees.csv")
df = pd.read_excel("employees.xlsx", sheet_name="Sheet1")
```

Useful `read_csv` options:

Option	Purpose	Example
`sep`	Different separator	`pd.read_csv("f.txt", sep="\t")`
`usecols`	Load only some columns	`usecols=["name", "age"]`
`nrows`	Load only the first n rows	`nrows=100`
`index_col`	Use a column as the index	`index_col="emp_id"`
`encoding`	Fix special-character errors	`encoding="latin-1"`

Saving data:

```
df.to_csv("output.csv", index=False)    # index=False avoids an extra index column
df.to_excel("output.xlsx", index=False)
```

1.5 Inspecting a DataFrame

These are the first commands to run on any new dataset.

Command	What it shows
`df.head(n)`	First n rows (default 5)
`df.tail(n)`	Last n rows (default 5)
`df.shape`	(rows, columns)
`df.size`	Total number of cells
`df.ndim`	Number of dimensions (2)
`df.columns`	Column names
`df.index`	Row labels
`df.dtypes`	Data type of each column
`df.info()`	Columns, non-null counts, dtypes, memory
`df.describe()`	Statistics of numeric columns
`df.nunique()`	Number of unique values per column

```
df.head(3)
df.shape          # (8, 6)
df.columns        # Index(['emp_id', 'name', 'department', 'city', 'age', 'salary'], dtype='object')
df.dtypes
```

```
emp_id         int64
name          object
department    object
city          object
age            int64
salary         int64
dtype: object
```

Tip: In pandas, text columns show as `object` (or `string` in newer versions).

`df.describe()` gives count, mean, std, min, 25%, 50%, 75% and max for numeric columns.

Use `df.describe(include="all")` to include text columns too.

`df.info()` is the best quick health check. It tells you how many non-null values each column has, which reveals missing data at a glance.

1.6 Key Points to Remember

- ✓ A DataFrame has **rows (index)** and **columns**.
- ✓ Index starts from **0** by default.
- ✓ `df.shape` and `df.columns` are attributes (no brackets); `df.head()` and `df.info()` are methods (with brackets).
- ✓ Always inspect with `head()`, `info()`, and `describe()` before analysis.

DAY 02

Data Selection and Filtering

2.1 Selecting Columns

```
df["name"]                # one column → returns a Series
df[["name", "salary"]]    # multiple columns → returns a DataFrame (note the double brackets)
```

Warning: Single brackets give a Series, double brackets give a DataFrame.

2.2 Selecting Rows and Columns with `loc` and `iloc`

	`loc`	`iloc`
Based on	**Labels** (names)	**Integer positions**
Slice end	**Included**	**Excluded**
Example	`df.loc[0:2]` → rows 0, 1, 2	`df.iloc[0:2]` → rows 0, 1

`loc[row, column]`

```
df.loc[0]                         # entire row with label 0
df.loc[0, "name"]                 # 'Asha'
df.loc[0:2, ["name", "salary"]]   # rows 0 to 2, two columns
df.loc[df["age"] > 30, "name"]    # names of people older than 30
```

`iloc[row_position, column_position]`

```
df.iloc[0]            # first row
df.iloc[0, 1]         # 'Asha' (row 0, column 1)
df.iloc[0:3, 0:2]     # first 3 rows, first 2 columns
df.iloc[-1]           # last row
df.iloc[:, 1:4]       # all rows, columns 1 to 3
```

Tip: Memory trick: **l**oc = **l**abel, **i**loc = **i**nteger position.

2.3 Filtering with Conditions (Boolean Masking)

A condition creates a True/False value for each row. Passing it to `df[...]` keeps only the True rows.

```
df["salary"] > 50000
# 0     True
# 1    False
# 2     True ...
df[df["salary"] > 50000]    # rows where salary is above 50000
```

```
   emp_id   name department       city  age  salary
0     101   Asha         IT  Bengaluru   28   60000
2     103  Meera         IT    Chennai   25   55000
5     106  Arjun         HR    Chennai   38   52000
6     107  Divya         IT       Pune   26   65000
```

2.4 Multiple Conditions

Operator	Meaning	Python keyword it replaces
`&`	AND	`and`
`|`	OR	`or`
`~`	NOT	`not`

```
# AND: IT department AND salary above 58000
df[(df["department"] == "IT") & (df["salary"] > 58000)]
# OR: Pune OR Chennai
df[(df["city"] == "Pune") | (df["city"] == "Chennai")]
# NOT: everyone except HR
df[~(df["department"] == "HR")]
```

Warning: **Always wrap each condition in parentheses `()`**, otherwise you get an error. Do not use `and` / `or` here; they don't work on Series.

2.5 Useful Filtering Methods

`isin()`: match any value in a list

```
df[df["city"].isin(["Pune", "Chennai"])]
df[~df["city"].isin(["Pune"])]            # NOT in
```

`between()`: value within a range (inclusive)

```
df[df["age"].between(26, 31)]
```

String filters with `.str`

```
df[df["name"].str.startswith("A")]         # names starting with A
df[df["name"].str.endswith("a")]           # names ending with a
df[df["name"].str.contains("an")]          # names containing 'an'
df[df["name"].str.contains("an", case=False)]   # ignore upper/lower case
```

Missing-value filters

```
df[df["salary"].isnull()]      # rows where salary is missing
df[df["salary"].notnull()]     # rows where salary is present
```

`query()`: SQL-like readable filtering

```
df.query("salary > 50000 and department == 'IT'")
min_age = 30
df.query("age > @min_age")     # use @ to reference a Python variable
```

2.6 Setting a Column as Index (Optional but Handy)

```
df2 = df.set_index("emp_id")
df2.loc[101]          # now you can select by employee ID
```

2.7 Key Points to Remember

- ✓ `df["col"]` → Series, `df[["col"]]` → DataFrame
- ✓ `loc` uses labels (end included), `iloc` uses positions (end excluded)
- ✓ Combine conditions with `&`, `|`, `~` and **use parentheses**
- ✓ `isin()` is the clean way to write many OR conditions on the same column

DAY 03

DataFrame Operations

3.1 Adding New Columns

```
# From a calculation
df["annual_salary"] = df["salary"] * 12
# From a constant
df["country"] = "India"
# From a condition: np.where(condition, value_if_true, value_if_false)
df["level"] = np.where(df["age"] > 30, "Senior", "Junior")
```

Multiple conditions with `np.select`:

```
conditions = [df["salary"] >= 60000, df["salary"] >= 45000]
choices    = ["High", "Medium"]
df["band"] = np.select(conditions, choices, default="Low")
```

3.2 Renaming Columns

```
df.rename(columns={"name": "employee_name", "salary": "monthly_salary"})
# Rename all columns at once
df.columns = ["ID", "Name", "Dept", "City", "Age", "Salary"]
# Clean-up trick: lowercase and replace spaces with underscores
df.columns = df.columns.str.lower().str.replace(" ", "_")
```

3.3 Dropping Columns and Rows

```
df.drop(columns=["country"])         # drop a column
df.drop(columns=["country", "band"]) # drop several
df.drop(index=[0, 1])                # drop rows by index label
```

Tip: `axis=1` means columns, `axis=0` means rows. `columns=` and `index=` are the clearer way to write it.

3.4 `inplace` vs Assigning Back

Most pandas methods return a **new** DataFrame and leave the original unchanged. To keep the change:

```
df = df.drop(columns=["country"])       # recommended: assign back
df.drop(columns=["country"], inplace=True)   # modifies df directly
```

3.5 Handling Missing Values

Create some missing data to practice:

```
df_m = pd.DataFrame({
    "name":   ["Asha", "Ravi", "Meera", "Kiran"],
    "age":    [28, np.nan, 25, 31],
    "salary": [60000, 45000, np.nan, np.nan]
})
```

Task	Code
Count missing per column	`df_m.isnull().sum()`
Show rows with any missing	`df_m[df_m.isnull().any(axis=1)]`
Drop rows with any missing	`df_m.dropna()`
Drop rows where a specific column is missing	`df_m.dropna(subset=["salary"])`
Fill with a fixed value	`df_m.fillna(0)`
Fill with the column mean	`df_m["age"].fillna(df_m["age"].mean())`
Fill with the median	`df_m["salary"].fillna(df_m["salary"].median())`
Forward fill (copy previous value)	`df_m.ffill()`

Tip: Which fill to choose? Mean for roughly symmetric numbers, median when there are outliers, mode (most frequent) for text/categories.

3.6 Handling Duplicates

```
df.duplicated()                      # True for repeated rows
df.duplicated().sum()                # count of duplicate rows
df.drop_duplicates()                 # remove duplicates, keep first
df.drop_duplicates(subset=["name"])  # check duplicates on selected columns only
df.drop_duplicates(keep="last")      # keep last occurrence instead
```

3.7 Changing Data Types

```
df["age"] = df["age"].astype(float)
df["emp_id"] = df["emp_id"].astype(str)
df["salary"] = pd.to_numeric(df["salary"], errors="coerce")   # bad values become NaN
```

3.8 Applying Functions: `apply`, `map`, `replace`

```python
# apply: run a function on every value of a column
df["tax"] = df["salary"].apply(lambda x: x * 0.1)
# apply with a normal function
def grade(sal):
    if sal >= 60000:
        return "A"
    elif sal >= 50000:
        return "B"
    return "C"
df["grade"] = df["salary"].apply(grade)
# map: replace values using a dictionary
df["dept_code"] = df["department"].map({"IT": 1, "HR": 2, "Sales": 3})
# replace: swap specific values
df["city"] = df["city"].replace("Bengaluru", "Bangalore")
```

Tip: For simple conditions, `np.where` is faster than `apply`. Use `apply` for more complex logic.

3.9 String Operations

```python
df["name"].str.upper()
df["name"].str.lower()
df["name"].str.len()
df["name"].str.strip()                # remove extra spaces
df["name"].str.replace("a", "@")
df["name"].str[0]                     # first letter
df["full"] = df["name"] + " - " + df["department"]    # combine columns
```

3.10 Resetting and Setting Index

```python
df.reset_index(drop=True)     # reset to 0,1,2... (drop=True discards the old index)
df.set_index("emp_id")        # make a column the index
```

Use `reset_index(drop=True)` after filtering or sorting so the index looks clean again.

3.11 Key Points to Remember

- ✓ New column: `df["new"] = ...`
- ✓ Methods return a new DataFrame, so **assign it back** (or use `inplace=True`)
- ✓ Check missing data with `isnull().sum()` before deciding to drop or fill
- ✓ `drop_duplicates()` and `astype()` are core cleaning steps

DAY 04

GroupBy and Aggregation

4.1 The Idea: Split → Apply → Combine

`groupby` works in three steps:

1. **Split** the data into groups (e.g. by department)
2. **Apply** a function to each group (mean, sum, count...)
3. **Combine** the results into one table

It is the same as `GROUP BY` in SQL.

```
Department   Salary                    Department   Mean Salary
IT           60000                     IT           60000
IT           55000    ──groupby──►     HR           48500
IT           65000                     Sales        43333.33
HR           45000
...
```

4.2 Basic GroupBy

```python
df.groupby("department")["salary"].mean()
```

```
department
HR       48500.000000
IT       60000.000000
Sales    43333.333333
Name: salary, dtype: float64
```

Other common aggregations:

```python
df.groupby("department")["salary"].sum()     # IT 180000, HR 97000, Sales 130000
df.groupby("department")["salary"].max()     # IT 65000, HR 52000, Sales 48000
df.groupby("department")["salary"].min()
df.groupby("department")["salary"].count()   # IT 3, HR 2, Sales 3
df.groupby("department")["salary"].median()
```

Function	Meaning
`sum()`	Total
`mean()`	Average
`median()`	Middle value
`min()` / `max()`	Smallest / largest
`count()`	Non-null values
`size()`	Rows in each group (includes nulls)
`nunique()`	Unique values
`std()`	Standard deviation

Tip: **`count()` vs `size()`:** `count()` ignores missing values, `size()` counts all rows.

4.3 Grouping by Multiple Columns

```
df.groupby(["department", "city"])["salary"].mean()
```

This creates groups for each (department, city) combination, such as IT + Pune and Sales + Bengaluru.

4.4 Multiple Aggregations with `agg()`

```
df.groupby("department")["salary"].agg(["min", "max", "mean"])
```

```
             min    max          mean
department
HR         45000  52000  48500.000000
IT         55000  65000  60000.000000
Sales      40000  48000  43333.333333
```

Different functions on different columns

```
df.groupby("department").agg({
    "salary": ["mean", "max"],
    "age": "mean"
})
```

Named aggregation (cleanest way)

```
result = df.groupby("department").agg(
    avg_salary=("salary", "mean"),
    max_salary=("salary", "max"),
    avg_age=("age", "mean"),
    employees=("emp_id", "count")
)
```

```
              avg_salary  max_salary  avg_age  employees
department
HR         48500.000000       52000     36.0          2
IT         60000.000000       65000     26.333333     3
Sales      43333.333333       48000     31.0          3
```

Format is `new_column_name=("existing_column", "function")`.

4.5 Turning the Result Back into a Normal DataFrame

A groupby result has the group name as its **index**. Use `reset_index()` to turn it into a regular column:

```
result = df.groupby("department")["salary"].mean().reset_index()
```

```
  department        salary
0         HR  48500.000000
1         IT  60000.000000
2      Sales  43333.333333
```

Or use `as_index=False`:

```
df.groupby("department", as_index=False)["salary"].mean()
```

4.6 Filtering Before and After GroupBy

```
# Filter first, then group: average salary of employees older than 27, per department
df[df["age"] > 27].groupby("department")["salary"].mean()
# Group first, then filter groups: only departments with average salary above 45000
avg = df.groupby("department")["salary"].mean()
avg[avg > 45000]
```

The second one is the pandas version of SQL's `HAVING`.

4.7 `transform()`: Keep the Original Shape

`agg` shrinks the table to one row per group. `transform` returns a value for **every original row**, which is handy for adding a group-level column:

```
df["dept_avg_salary"] = df.groupby("department")["salary"].transform("mean")
df["diff_from_avg"] = df["salary"] - df["dept_avg_salary"]
```

4.8 Pivot Tables

A pivot table summarizes data in a grid, like an Excel pivot table.

```
pd.pivot_table(
    df,
    values="salary",
    index="department",     # rows
    columns="city",         # columns
    aggfunc="mean",
    fill_value=0            # replace empty cells
)
```

Add totals with `margins=True`.

4.9 `crosstab()`: Count Combinations

```
pd.crosstab(df["department"], df["city"])
```

This counts how many employees fall into each department and city combination.

4.10 SQL ↔ Pandas Comparison

SQL	Pandas
`SELECT department, AVG(salary) FROM emp GROUP BY department`	`df.groupby("department")["salary"].mean()`
`COUNT(*)`	`size()`
`HAVING AVG(salary) > 45000`	Filter on the groupby result

4.11 Key Points to Remember

- ✓ `groupby` = split → apply → combine
- ✓ Use `agg()` with **named aggregation** for clean, readable column names
- ✓ `reset_index()` converts the result back to a normal table
- ✓ `transform()` keeps the original number of rows
- ✓ `pivot_table` is Excel's pivot table in Python

DAY 05

Sorting and Data Analysis

5.1 Sorting by Values

```
df.sort_values("salary")                      # ascending (low → high), default
df.sort_values("salary", ascending=False)     # descending (high → low)
```

Sort by multiple columns

```
df.sort_values(["department", "salary"], ascending=[True, False])
```

This sorts by department A→Z, and **within each department** by salary high→low.

Other useful options

```
df.sort_values("salary", na_position="first")   # put missing values first (default is last)
df.sort_values("salary", ignore_index=True)     # reset the index after sorting
```

Age sorted ascending gives: Meera (25), Divya (26), Asha (28), Sneha (29), Kiran (31), Rahul (33), Ravi (34), Arjun (38).

5.2 Sorting by Index

```
df.sort_index()                     # sort by row labels
df.sort_index(ascending=False)
df.sort_index(axis=1)               # sort column names A→Z
```

5.3 Top N and Bottom N

```
df.nlargest(3, "salary")      # top 3 highest salaries
df.nsmallest(3, "salary")     # 3 lowest salaries
```

```
   emp_id   name department       city  age  salary
6     107  Divya         IT       Pune   26   65000
0     101   Asha         IT  Bengaluru   28   60000
2     103  Meera         IT    Chennai   25   55000
```

Tip: `nlargest()` is shorter and faster than `sort_values(...).head(n)`.

5.4 Ranking

```
df["salary_rank"] = df["salary"].rank(ascending=False)
df["salary_rank"] = df["salary"].rank(ascending=False, method="dense")   # no gaps after ties
```

Rank inside each group:

```
df["rank_in_dept"] = df.groupby("department")["salary"].rank(ascending=False)
```

5.5 Counting Values

```
df["city"].value_counts()
```

```
city
Pune         3
Bengaluru    3
Chennai      2
Name: count, dtype: int64
```

```
df["city"].value_counts(normalize=True)    # proportions (0 to 1)
df["city"].value_counts(ascending=True)
df["city"].nunique()                       # 3 unique cities
df["city"].unique()                        # array of unique values
```

5.6 Summary Statistics

```
df["salary"].mean()        # 50875.0
df["salary"].median()
df["salary"].mode()
df["salary"].min()
df["salary"].max()
df["salary"].sum()         # 407000
df["salary"].std()         # standard deviation
df["salary"].var()         # variance
df["salary"].quantile(0.75)      # 75th percentile
df["age"].mean()           # 30.5
```

Full summary in one line:

```
df.describe()
```

5.7 Finding the Row of Max / Min

```
df["salary"].idxmax()              # index of highest salary → 6
df.loc[df["salary"].idxmax()]      # the full row (Divya)
df.loc[df["salary"].idxmin()]      # the full row (Kiran)
```

5.8 Correlation

Correlation shows how strongly two numeric columns move together (from -1 to +1).

```
df[["age", "salary"]].corr()
df.corr(numeric_only=True)       # all numeric columns
```

Value	Meaning
Close to **+1**	Both increase together
Close to **-1**	One increases, the other decreases
Close to **0**	No linear relationship

> **Warning:** Correlation does not prove one thing **causes** the other.

5.9 Cumulative and Percentage Calculations

```
df["cumulative_salary"] = df["salary"].cumsum()
df["salary_pct"] = (df["salary"] / df["salary"].sum() * 100).round(2)
```

5.10 Mini Analysis Workflow (Putting Days 1–5 Together)

```
# 1. Look at the data
df.head()
df.info()
# 2. Clean
df = df.drop_duplicates()
df["salary"] = df["salary"].fillna(df["salary"].median())
# 3. Filter
it_team = df[df["department"] == "IT"]
# 4. Group and aggregate
summary = df.groupby("department").agg(
    employees=("emp_id", "count"),
    avg_salary=("salary", "mean"),
    max_salary=("salary", "max")
).reset_index()
# 5. Sort
summary = summary.sort_values("avg_salary", ascending=False)
# 6. Answer a question
print("Highest paying department:", summary.iloc[0]["department"])   # IT
```

Typical questions you can now answer:

- Which department pays the most on average? → IT
- Who are the top 3 earners? → `nlargest(3, "salary")`
- Which city has the most employees? → `value_counts()`
- How many employees per department? → `groupby().count()`

5.11 Key Points to Remember

- ✓ `sort_values()` for values, `sort_index()` for index
- ✓ `nlargest()` / `nsmallest()` for quick top or bottom N
- ✓ `value_counts()` is the fastest way to see category frequency
- ✓ `describe()`, `corr()`, and `idxmax()` give quick insights
- ✓ Clean → Filter → Group → Sort is the standard analysis flow

Quick Cheat Sheet

Task	Code
Read CSV	`pd.read_csv("file.csv")`
First 5 rows	`df.head()`
Shape	`df.shape`
Info / Stats	`df.info()`, `df.describe()`
Select columns	`df[["a", "b"]]`
By label / position	`df.loc[...]`, `df.iloc[...]`
Filter	`df[(df["a"] > 5) & (df["b"] == "x")]`
In a list	`df[df["a"].isin([1, 2])]`
New column	`df["new"] = df["a"] * 2`
Conditional column	`np.where(cond, "yes", "no")`
Rename	`df.rename(columns={"old": "new"})`
Drop column	`df.drop(columns=["a"])`
Missing values	`isnull().sum()`, `fillna()`, `dropna()`
Duplicates	`drop_duplicates()`
Change type	`astype()`
Apply function	`df["a"].apply(func)`
Group and aggregate	`df.groupby("a").agg(x=("b", "mean"))`
Pivot table	`pd.pivot_table(df, values, index, columns, aggfunc)`
Sort	`df.sort_values("a", ascending=False)`
Top N	`df.nlargest(3, "a")`
Frequency	`df["a"].value_counts()`
Correlation	`df.corr(numeric_only=True)`

Practice Questions

Try these on the sample dataset (answers can be checked by running your code).

Day 01

1. Show the shape, column names, and data types of the DataFrame.
2. Display the last 3 rows.

Day 02

3. Select only `name` and `city` columns.
4. Find employees from Pune with salary above 45000.
5. Find employees whose name starts with "A" or "D".

Day 03

6. Add a column `bonus` equal to 10% of salary.
7. Add a column `experience_level` that is "Senior" if age > 30, else "Junior".
8. Rename `salary` to `monthly_salary`.

Day 04

9. Find the average age per city.
10. Find the number of employees and total salary per department.
11. Create a pivot table of average salary by department and city.

Day 05

12. Sort employees by salary (highest first).
13. Find the top 2 earners in each department.
14. Which city has the most employees?
15. Find the correlation between age and salary.

About This Repository

These notes are part of my **30-Days-of-Data-Analysis-Challenge** (Python, MySQL, Excel).

If you found them helpful, give the repo a star and follow along for the next days!

Next up: Day 06 onwards